Unit 2

Regular Expressions

Regular set : Regular sets are set which are accepted by FA (Finite Automata).

ex: L = {ε, 11, 1111, 111111, ...} Regular Expression are used for representing certain sets of strings in an algebraic fashion.

⇒ Strings of an algebraic fashion

Closure Properties of Regular sets :

Rules :

empty NULL symbol.

1. Any terminal symbol i.e symbols ∈ (belongs to) Σ, Λ and Φ are regular expression
2. The union of two regular expression is also a regular expression
3. The concatenation of two regular expression is also a regular expression
4. The iteration (or closure) of a regular expression is also a regular expression
5. The regular expression over Σ are precisely those obtained recursively by the application of the above rules once or several time.

1. ex: a, b, c, Λ, Φ terminal symbol,
2. R1, R2 (union) (R1 + R2) — regular.
3. R1, R2 R1.R2 — Regular.
4. R → R* R → 1 $a^* = Λ, a, aa, aaa$

$a^* = Λ, a, aa, aaa$. — closure.

Any set that represents the value of the Regular Expression is called a Regular set

Sets as Regular Expression ②

- Regular Expression - Examples

Describe the following sets as Regular Expression

1. {0, 1, 2}
   R = 0 + 1 + 2
   0 or 1 or 2
   + represents OR

2. {^, ab}
   R = ^ ab
   empty symbol only

③ {abb, a, b, bba} abb or a or b or bba.

R = abb + a + b + bba

④ {^, 0, 00, 000, ...} closure of 0

String from by using Any number of zero. [Represented by * symbol]

R = 0* → zero closure

⇒ closure is always with ^ symbol only

5) {1, 11, 111, 1111, ...}

one closure.

R = 1+ → closure of symbol not denoted with ^ symbol.

→ Represented by + symbol.

Cross

Identities of Regular Expression

1) $\phi + R = R$
2) $\phi R + R\phi = \phi$
3) $\epsilon R = R\epsilon = R$.
4) $\epsilon^* = \epsilon$ and $\phi^* = \epsilon$
5) $R + R = R$.
~~$R^* R^* = R^*$~~
6) $R^* R^* = R^*$
7) $RR^* = R^* R$
8) $(R^*)^* = R^*$
9) $\epsilon + RR^* = \epsilon + R^* R = R^*$
10) $(PQ)^* P = P(QP)^*$.
11) $(P+Q)^* = (P^* Q^*)^* = (P^* + Q^*)^*$
12) $(P+Q)R = PR + QR$ and
$R(P+Q) = RP + RQ$.

closure Properties

A set is closed under an operation if applying that operation to any members of the set always yields a member of the set.

For ex: The posible integers are closed under addition and multiplication but not division.

ex°. When we add two integers we will get Answer as 1+2 = (3) that 1 & 2 are the membend third integer. Hence 3 is also called as closure property set & a membend set.

An Example :-
Proof using Identities of Regular Expression

ex.1 Prove that :- $(1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)$ is equal to $0^*1(0+10^*1)^*$

Soln

$$LHS = (1+00^*1) + (1+00^*1)(0+10^*1)^*(0+10^*1)$$
$$= (1+00^*1)[\epsilon + (0+10^*1)^*(0+10^*1)]$$
$$= (1+00^*1)(0+10^*1)^* \quad [\epsilon + R^*R = R^*]$$
$$= (\epsilon 1 + 00^*1)(0+10^*1)^* \quad [\epsilon . R = R]$$
$$= (\epsilon + 00^*)1(0+10^*1)^*$$
$$= 0^*1(0+10^*1)^* = RHS$$

Hence proved

Manipulation of Regular Expressions
or
Designing Regular Expressions

Design Regular expression for the following languages over {a, b}

1) Language accepting strings of length exactly 2.
2) Language accepting strings of length atleast 2
3) Language accepting strings of length atmost 2.

(1)

Sol? $\Sigma = \{a, b\}$.

$L_1 = \{aa, ab, ba, bb\}$

$R = aa + ab + ba + bb$

[just do the union of all the strings possible in set]

$= a(a+b) + b(a+b)$

$= (a+b)(a+b)$

when length 2. i.e. $(a+b)(a+b)$

When we want length exactly 3 then we have to add one more $(a+b)$.

& for 4 again added $(a+b)$.

Now Design the Regular Expression

2) Atleast 2 lengths. means 2 or more

$L_1 = \{aa, ab, ba, bb, aaa, \ldots\}$
Infinite set.

$R = (a+b)(a+b)$ ⟶ This is used to represent the exactly length 2 from previous example.

$R = (a+b)(a+b)(a+b)^*$
↓ It represents more than length 2.

Now we can generalize it as suppose length 2 we solve the above equation by ~~replacing~~ Replacing * with 0.

$R = (a+b)(a+b)(a+b)^0$

when it is replaced with 0 it takes 1. so we get soln as:

$R = (a+b)(a+b)(1)$

when we replace * as 1 we get the soln for string having length 3.

$R = (a+b)(a+b)(a+b)^1$

$R = (a+b)(a+b)(a+b)$

Suppose we want to get length 4 string, we have to replace 'a' with 4., so we get-

$R = (a+b)(a+b)(a+b)$

~~In this we get the sol~~

So it can accept any string over length 2, atleast 2.

(3) Atmost 2 ⇒ maximum 2 & minimum nothing
⇒ not more than 2.

$L1 = \{a, b, aa, ab, ba, bb\}$

$L1 = \{\epsilon, a, b, aa, ab, ba, bb\}$
string accepted by language. Union of strings.

$R = \epsilon + a + b + aa + ab + ba + bb$

$R = (\epsilon + a + b)(\epsilon + a + b)$

simply & write it as

$R = (\epsilon + a + b)(\epsilon + a + b)$

Expression for Atmost 2

Because, suppose we want to design for length 0, so we pull only ϵ value from the Regular Expression & just discard the value of $(a+b)$.

$R = \epsilon + (a+b)(\epsilon + a+b)$ ✓ ✗ ✗

R = only ϵ is considered.

2) if we want string of length 1.
Then we have to consider only 'a' or 'b'
from the Regular Expression.

$R = (\epsilon + a + b)(\epsilon + a + b)$

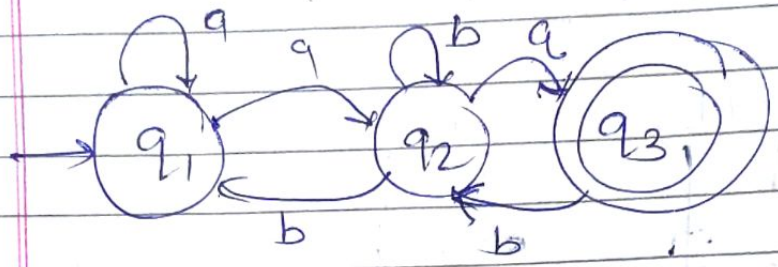
a from here — choose a from here (a or a)
b from here — choose b from here (b or b)

for the above string we can accept
it as for aa.
we accept
a from both the Expression.

$R = (\epsilon + a + b)(\epsilon + a + b)$

Atmost two.

ex:2 Find the Regular Expression for Given NFA.



Sol? ① First we have to write equations for each of state of NFA

② After that we have to put the equations of all the state to the final state & then we get the Regular expression for NFA.

for writing equation check incoming transitions

$q_3 = q_2 a$ ——— ①

it coming from q_2 with input a so we write it as it is $q_2 a$ ⇒ only one incoming transition

consider only incoming transitions

q_2 = As consider q_2 state there are three incoming transactions are there so we can consider all of them, & use union operation on them so we get.

$q_2 = q_1 a + q_2 b + q_3 b$ ——— ②

q_1 = When we consider initial state coming from nowhere indicate 'ε' so we put equation as

$q_1 = \epsilon + q_1 a + q_2 b$ ——— (3).

we have to simplify it

(1) ⇒ $q_3 = q_2 a$. replace q_2 from eq. (2)

$q_3 = (q_1 a + q_2 b + q_3 b) a$.

$q_3 = q_1 aa + q_2 ba + q_3 ba$ ——— (4).

(2) $q_2 = q_1 a + q_2 b + q_3 b$

we have to substitute value of q_3 from equation (1).

Now putting value of q_3 from eq (1). What we get-

$q_3 = q_1 a + q_2 b + (q_2 a) b$

$= q_1 a + q_2 b + q_2 ab$

$= q_1 a + q_2 (b + ab)$

equals the form $R = Q + RP$

$R = QP^*$

Arden's Theorem

$q_2 = q_1 a + q_2 (b + ab)$

R Q R P

$q_2 = (q_1 a)(b + ab)^*$ ——— (5)

(3) $q_1 = \epsilon + q_1 a + q_2 b$.

putting value of q_2 from eq (5)

$q_1 = \epsilon + q_1 a + (q_1 a)(b + ab)^* b$

$q_1 = \epsilon + q_1 (a + a(b + ab)^*) b$

R Q R P

$R = Q + RP$

$R = QP^*$

$$q_1 = \epsilon((q+1a(b+qb)^*)\,b)^*$$

From identities,

$\epsilon \cdot R = R$.

$$q_1 = (q + q\,(b+qb)^*\,b)^* \longrightarrow (6).$$

Final state (q_3)

$q_3 = q_2 q$ ——— from eq (1)

replace q_2 from eq (5)

$= q_1 q (b+qb)^* q$ [from eq (5)

replace q_1 from eq (6)

$q_3 = (q+q(b+qb)^*b)^*\, a\,(b+qb)^*\, q$

By putting value of q_1 from eq (6)

q_3 = Required Regular Expression for given NFA.

Conclusion

1. Equating all Each State
2. Simplify the equations by Arden's theorem or identities or simplifying techniques
3. Finally we take all the equations & for the final state, we substitute value of the each equations & other states and finally we
4. Finally we get the expression in terms of q & b which is expressed completely in terms ~~of q & b~~ of input symbols and that is nothing but the regular expressions.

Arden's theorem

If P an Q are two Regular Expressions over Σ, and if P does not contain ϵ, then the following equation in R given by $R = Q + RP$ has a unique solution i.e $R = QP^*$

$R = Q + RP$

$R = QP^*$ ← Arden's theorem

when we get the equation $R = Q + RP$ we can directly replace it with $R = QP^*$ ← Because it is sol? by Arden's theorem

$R = Q + RP$ ——— (1)

replace $R = QP^*$

$= Q + QP^*P$

$= Q(\epsilon + P^*P)$ from Identities $\epsilon + R^*R = R^*$

$= QP^*$

Hence proved that $R = QP^*$ is the sol? for $R = Q + RP$.

$R = Q + RP$

$= Q + [Q + RP]P$

$= Q + QP + RP^2$

replace R with $Q + RP$

$= Q + QP + [Q + RP]P^2$

$= Q + QP + QP^2 + RP^3$

n times repeated rep

$= Q + QP + QP^2 + QP^n + RP^{n+1}$

$= Q + QP + QP^2 + \ldots + QP^n + RP^{n+1}$

replace $[R = QP^*]$

$= Q + QP + QP^2 + \ldots QP^n + QP^*P^{n+1}$

$= Q[\epsilon + P + P^2 + \ldots P^n + P^*P^{n+1}]$

closure of $P = P^*$

$\boxed{R = QP^*}$

Hence proved this is the only sol. of Arden's theorem.

Assign

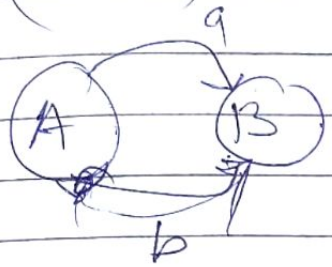
Page: 14
Date:

Conversion of ~~Equivalence bet 2~~ Regular Expression to Finite Automata

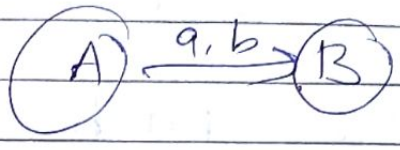
Conversion :-

Basic

(a+b)



or



(a.b)

(ab)



a^*

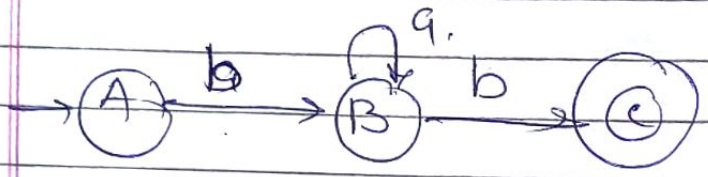


Basic Conversion of Regular Expression to Finite Automata.

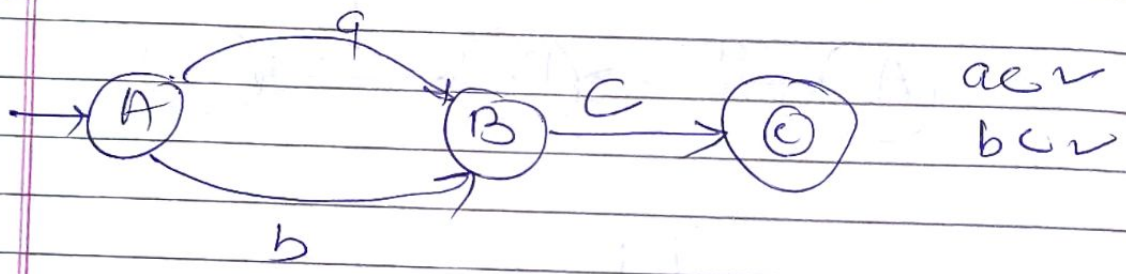
ex:1 Construct the following Regular Expression to their equivalent Finite Automata

1) ba^*b
2) $(a+b)c$
3) $a(bc)^*$

Sol: ba^*b $\quad L = \{bb, bab, baab, \ldots\}$



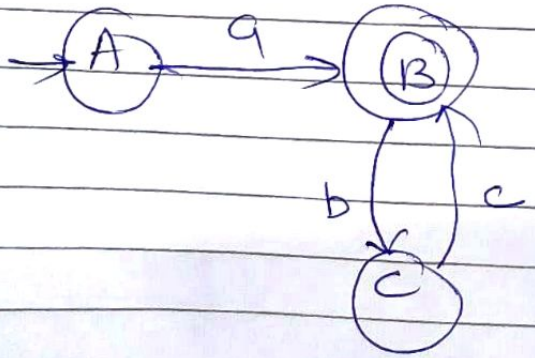
(2) $(a+b)c$



ac ✓
bc ✓

(3) $a(bc)^*$

$L = \{a, abc, abcbc, abcbcbc\}$



Equivalence RE & FA

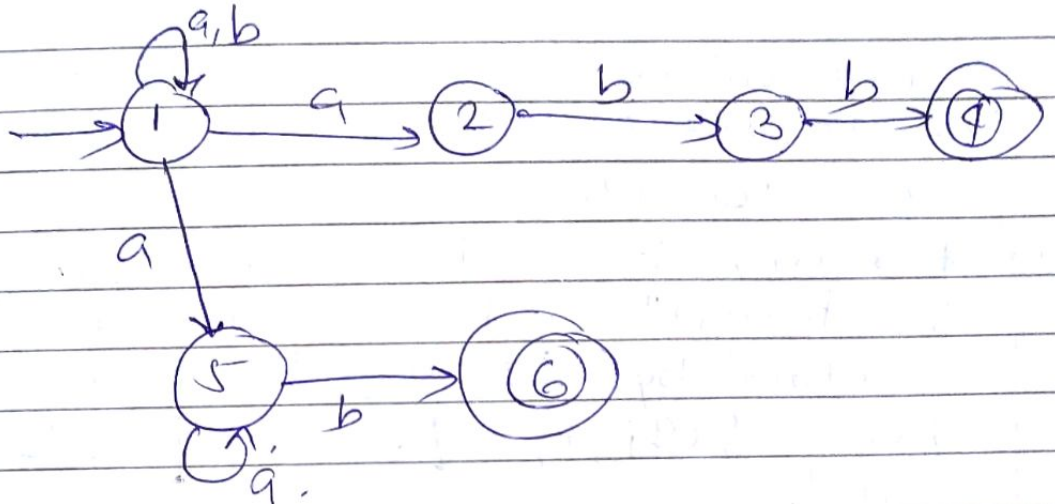
Ex:2 ~~Construct~~ Convert the following Regular Expression to its equivalent finite Automata.

$(a|b)^*(abb|a^*b)$

Soln

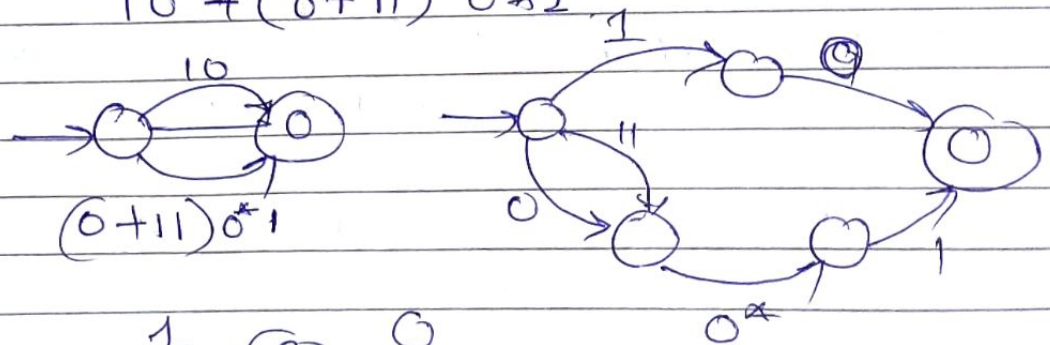
$a^+ = \{a, aa, aaa\}$

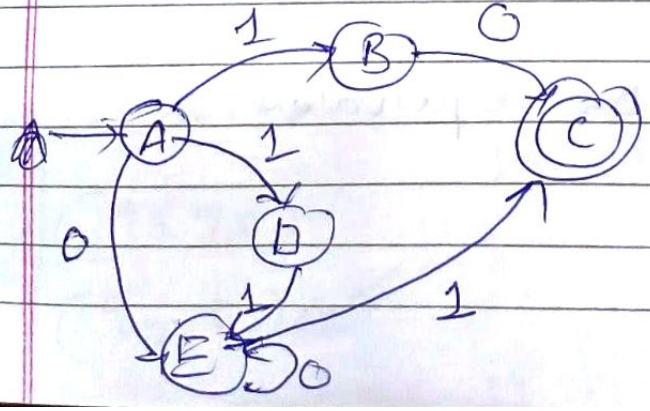
$a^* = \{\epsilon, a, aa\}$



Ex:3 Convert the following Expression to its equivalent finite Automata.

$10 + (0+11)\,0^*1$





Ex: Equivalence betn ~~RE and~~ FA

Regular Expression & finite Automata is same or not-

Two automaton are equivalent means, Two automata performs same ~~task~~ function i.e languages accepted or identified by two automatas are same.

Equivalence of two finite Automata. There are certain steps.

Steps to identify equivalence.

1) ~~for~~ any pair of states $\{q_i, q_j\}$ the transition for input $a \in \Sigma$ (Belong to) is defined by $\{q_a, q_b\}$ where $\delta(q_i, a) = q_a$ and, $\delta\{q_j, a\} = q_b$.

Two automata are not equivalent if for a pair $\{q_a, q_b\}$ one is INTERMEDIATE state and the other is FINAL state.

2) If initial state is final state of one automaton, then in second automaton also initial state must be final state for them to be equivalent.

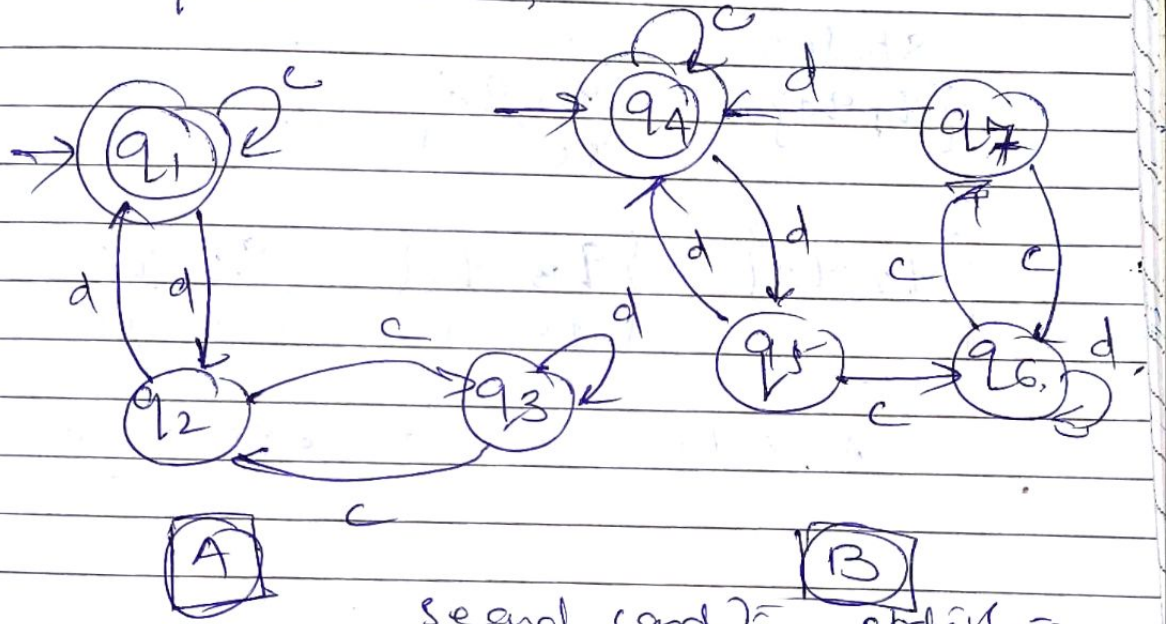
Select the pairs of sets.

q_i goes to q_a } on getting input a
and q_i goes to q_b. }

when q_a & q_b are in same state either in final or Intermiade state two Automata are equivale.

But one in final & one in Intermd step. then that two Automata is not in equivalence,

ex:



second condition satisfies.

In both of them Initial & Final state same,

We make table (Inputs & States)

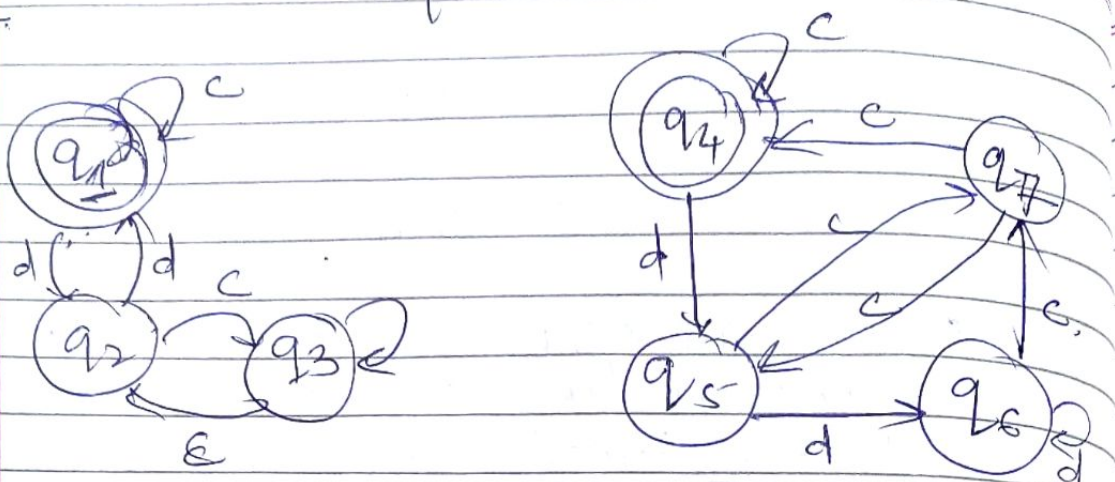
States	c	d.	
(q_1, q_4)	(q_1, q_4) FS FS	(q_2, q_5) IS IS	Intermediate state
$(q_2 q_5)$	$(q_3\ q_6)$ IS IS	$(q_1\ q_4)$ FS FS	
$(q_3\ q_6)$	(q_2, q_7) IS IS	$(q_3\ q_6)$ IS IS	
(q_2, q_7)	(q_3, q_6) IS IS	(q_1, q_4) FS FS	

FS Final state

A & B are equivale

Find out whether automata are equivalent or not.

ex: 2



A Automata

B Automata

states	c	d
$\{q_1, q_4\}$	$\{q_1, q_4\}$ FS FS	$\{q_2, q_5\}$ IS IS
$\{q_2, q_5\}$	$\{q_3, q_7\}$ IS IS	$\{q_1, q_6\}$ FS FS

A & B are not equivalent.

Pumping Lemma

If A is Regular language then ~~then it must~~ has to have Pumping Lemma ~~with~~ take place

~~If~~ A is Regular it should be able to pumped.

Pumping Lemma For RE

Regular Expression - Language.

→ Pumping lemma is used to prove that Language is not regular.

⇒ Regular languages are that used to design finite state machine.

→ Due to limitations some Languages that cannot be designed by finite state machine - Non Regular Language

⇒ How can we prove that the language is non Regular.

⇒ For That we are using a Pumping Lemma

Pumping lemma:

Pumping lemma is used to prove that the language is not Regular.

But it cannot be used to prove that a language is Regular.

Statement of Pumping Lemma:

If A is Regular Language, then A has a Pumping Length 'p' such that any string 's' where $|s| \geq p$ may be divided into 3 parts $s = xyz$ such that the following conditions must be true:

1) $xy^iz \in A$ for every $i \geq 0$ (∈ Belongs to.)
2) $|y| > 0$
3) $|xy| \leq p$

Assume;

- A is Regular Language.
- S is the string belays to A.
- A integer P is the pumping length.
  when the bestring is greater or equal to P.
- $|S| \geq P$ divided into XYZ. 3 parts.

consider $XY^iZ \in A$ for every $i \geq 0$.

— so, we consider

→ let x & z keep as it is.

→ middle part may changed will increase it same number of times as $i \geq 0$. then also the string after increasing part also belays to A.
& the string that we obtained. Also belays to A.

| Y = same length of symbols | Sequence of sym

⇒ Three conditions must be prove for pumping lemma

1) $XY^iZ \in A$ for every $i \geq 0$

2) $|Y| > 0$ greater than zero.

3) $|XY| \leq P$ less than or equal to P.

To prove that a Language is not Regular using PUMPING LEMMA, follow the below steps :

(We prove using Contradiction)

- Assume that A is Regular.
- It has to have a Pumping Length (Say P)
- All strings longer than P can be pumped.
  $|s| \geq P$
- Now find a string 's' in A such that $|s| \geq P$
- Divide s into xyz
- Show that $xy^iz \notin A$ for some i
  (Not below)
- Then consider all ways that s can be divided into xyz.
- Show that none of these can satisfy all the 3 pumping conditions at the same time
- S cannot be Pumped == Contradiction

CONTRADICTION

A Regular — not Regular

①

Ex 8 Using Pumping Lemma prove that the language $A = \{a^n b^n \mid n \geq 0\}$ is Not Regular.

Sol^n We know that,
finite state machine cannot keep count of anything.
Simple
Limited memory

$A = \{a^n b^n \mid n \geq 0\}$

Why this is not regular language?
~~are~~ As we know tho ~~Because~~
~~In this case same no. of a' are be followed by b's~~ same no of b's
~~In order to achieve Regular~~ Language
Because
First of all we have to find out How many a's are there. Find out the no & same no of b's are to be followed.
We can repeat same no. b's after finding out no of a's
We know that finite state machine cannot keep A/c of anything
Limited memory
So this language cannot design for finite state machine, that is not Not Regular Language.
To prove this, we use Pumping Lemma.

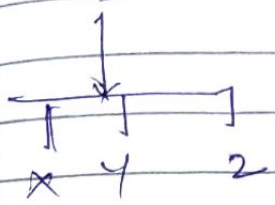
Refer to the Rules of Pumping Lemma :-

We Assume A is Regular Language

Need to have P is pumping length. = P [Any string which is obtained by replacing pumped]

$S = a^P b^P$. → P is pumping len.

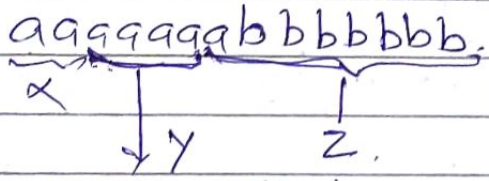
$S = a^P b^P$. Divide S into x y z



⇒ S = aaaaaaa bbbbbbb Stop.

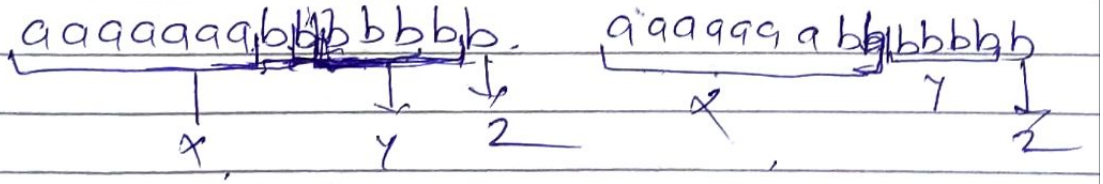
Assume P = 7

Case 1 ; y part is in the 'a' part.

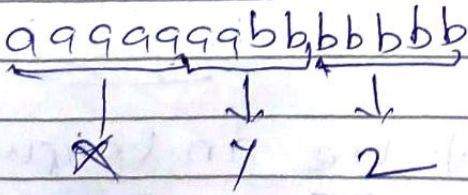


Assume this is the y part.

Case 2 ; The y is in the 'b' part.



Case 3 : The y is in 'a' & 'b' part



$xy^i z \notin A$

Not belap to A.

Case 1 $xy^i z \Rightarrow xy^2 z$

Assume $i=2$ y part repeat twice

aaaaaa aaaa abbbbbbb

11 a's 7 b's

a's ≠ b's.

No of a's not equal to No of b's

Does not lie in our language

as our language

condition says

no of a's & b's

2) $xy^i z \Rightarrow xy^2 z$

$i=2$

aaaaaaabb bbbbbbbb.

7 a's ≠ 11 b's

3) $xy^i z \Rightarrow xy^2 z$

$i=2$

aaaaa aabb aabb bbbbb.

Before count of a's & b's. ⇒ This is not followed the pattern $a^n b^n$ lie in

Here it not a Regular Language

Does not lie in language

it

Shown readily is not belap to A

(2) $|y| > 0$ satisfied

$i = 2$.

xy^2z pattern

does not follow the language A.

So

condition not satisfied,

(3) $|xy| \leq p$

In this case $p \geq 7$.

case 1 = 6. satisfies.

case 2 = 13. Not-satisfied

case 3 = 9 Not-satisfies.

As the conditions are not satisfied

Thus, cannot pumped, then Language is not Regular

Ex 2 Using Pumping lemma, prove that the language. $A = \{yy \mid y \in \{0,1\}^*\}$ is not Regular.

Assume A is Regular | $\{0,1\}^* = 0101$ ⇒ 01 string repeat

Pumping Length p.

Choose $s = 0^p 1 0^p 1 = 0^p 1 0^p 1$.

$s = \underbrace{000000}_{x}\underbrace{0}_{y}\underbrace{1000000}_{z}1$

$p = 7$

$xy^2z \Rightarrow xy^2z$. $i = 2$

00000000010000001 $\notin A$

Pattern not.

Thus Not-Regular belong to A

Thus Language is not Regular

$|y| > 0$ satisfies.

$|xy| \leq p$ satisfies

$6 \leq p$ $\Rightarrow s \notin A$

$xy \mid y \in \{0,1\}^*\}$

First part of string is equal to the second part of string.

Ans) $Y = 01$ First half
Second, $Y = 01$ Second half.

0101

Finite state machine is limited memory & in this ~~one~~ Language first part 01 is repeated as second part 01, to do need memory & it is not possible for finite state machine. Hence Not Regular language

RA13

Page: 28
Date:

Regular Grammar

What is Grammar?

Normal Human ~~Rule~~ Languages:

Grammar is nothing but the set of Rules used for proper conversation with each other.

Likewise

In computer languages:

There ~~are some~~ mathematical model ~~Rules which~~ are used to write. of Grammar which is used to write computer languages.

For computer languages, there is a mathematical model of grammar used to write computer languages.

⇒ Noam chomsky gave a mathematical model of Grammar which is effective for writing computer languages.

According to Noam Chomsky there are four types of Grammar

Grammar Type	Grammer Accepted	Langage Accepted	Automaton
TYPE-0	Unrestricted grammar	Recursively Enumerable	Turing machine
TYPE-1	Context sensitive Grammer	Context sensitive Language	Linear Bounded
TYPE-2	Context Free Grammer	context free Language	Pushdown Automata
TYPE-3	Regular Grammar	Regular Language	Finite state Automaton

Grammar :

How can we find Grammar?

→ A Grammar 'G' can be formally described using 4 tuples as $G = (V, T, S, P)$ where,

V = set of Variables or Non Terminal symbols.

T = set of Terminal Symbols

S = Start symbol.

P = Production rules for Terminals and Non-Terminals.

A production rules has the form $\alpha \to \beta$ where α and β are strings of $V \cup T$ and atleast one symbol of α belongs to V.

Example $G = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to a, B \to b\})$

V union T

Belong to set of Non terminal symbols & Terminal symbols.

$G = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to a, B \to b\})$

$V = \{S, A, B\}$

$T = \{a, b\}$

$S = S$.

$P = S \to AB, A \to a, B \to b$

Example:

$S \to AB$

$\to aB$

$S \to ab$

String designed by using Grammer.

Regular Grammar :

Regular Grammar is of two types.

1) Right Linear Grammar
2) Left Linear Grammar.

Right Linear Grammar :

- A Grammar is said to Right Linear if all productions are of the form

$A \to xB$ | Before B: Non Terminal symbol B lies right side of T.Ter
$A \to x$

where $A, B \in V$ A, B Belongs to V
and $x \in T$ x Belongs to T

Terminal Symbols lies to right of side of Terminal symbol

Left Linear Grammar :

- A Grammar is said to be Left Linear. if all production are of the form.

$A \to Bx$ After B
$A \to x$

where $A, B \in V$ A, B belongs to V
and $x \in T$ x belongs to T.

If Non Terminal symbols lies to left of Terminal Symbol

eg: $S \to abS \mid b$ — Right Linear Grammar
$S \to Sb \mid b$ — Left Linear Grammar

where, $a, b \in T$

$a, b \in T$	$S \in V$
$b \in T$	Non Terminal symbol V
$S \in V$	Terminal sym T

Inter Conversion of RG to RE

Derivations from a Grammar

- The set of all strings that can be derived from a Grammar is said to be the LANGUAGE generated from the grammar.

Example 1: consider the Grammar

$G1 = (\{S, A\}, \{a, b\}, S, \{S \to aAb, aA \to aaAb, A \to \epsilon\})$.

$S \to aAb$ [by $S \to aAb$.

$\to aaAbb$ [by producing the rule. $aA \to aaAb$]

$\to aaaAbbb$ [by $aA \to aaAb$]

$\to aaa\epsilon bbb$ [ϵ means leave it blank]

$\to aaabbb$ [by $A \to \epsilon$]

Result: Language generated from this Grammar is aaabbb. $L(G1) = \{aaabbb\}$

ex:2 $G2 = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to a, B \to b\})$

$S \to AB$ This is the only string generated by the grammar.

$\to ab$

$L(G2) = \{ab\}$.

ex:3 $G3 = (\{S, A, B\}, \{a, b\}, S, \{S \to AB, A \to aA \mid a, B \to bB \mid b\})$

(1) $S \to AB$ A belongs to aA and a also. Let me first take small a.

$\to ab$ aB and b

(2) $S \to AB$
$\to aAbB$
$\to aabb.$

Now we take A as small a.
B as small B

(3) $S \to AB$
$\to aAb$
$\to aab.$

Now take A as aA & B as b.

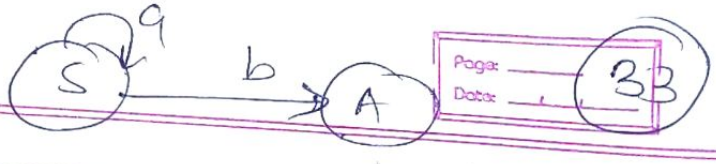
(4) $S \to AB$
$\to aBb$
$\to abb$

Now take A as small a
& B as Bb.
Then B replace with b.

Depends on choice we can generate various strings,
so language generated as
$L(ab) = \{ab, aabb, aab, abb\}$ minimum.
$L(ab) = \{ab, a^2b^2, a^2b, ab^2, \ldots\}$ generalized.

$$L(ab) = \{a^n b^m \mid m \geq 0 \text{ and } n \geq 0\}$$

Language generated from derived from given grammer.



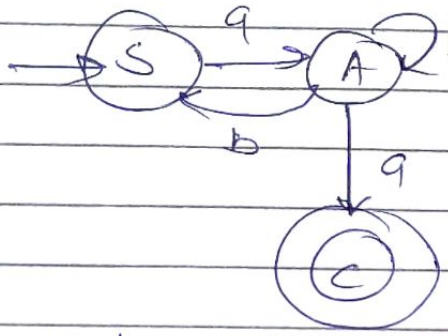
Conversion of RLG to LLG:

$S \rightarrow aA$

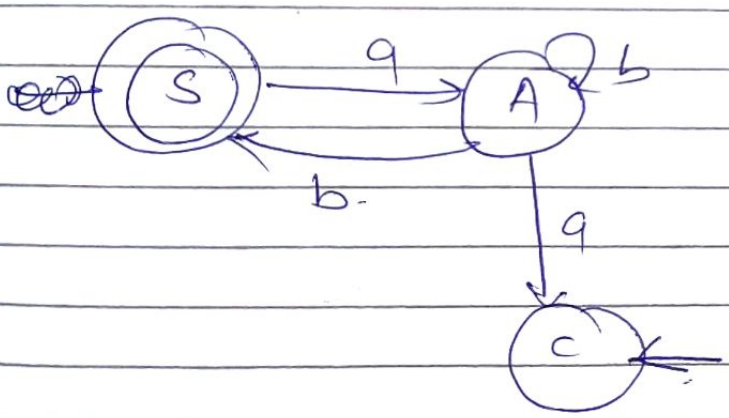
$A \rightarrow bA \mid bS \mid a$

$X \rightarrow T^*V$ or $X \rightarrow T^*$

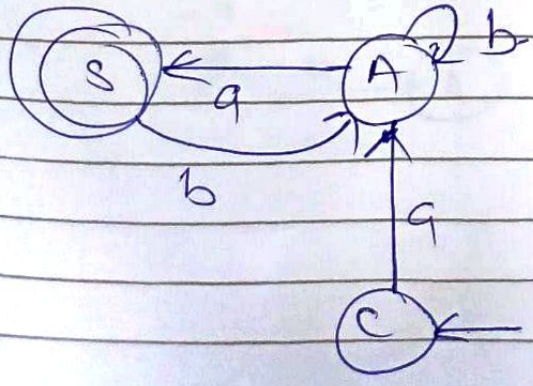
Step 1: Convert the given Right Linear grammer to FA.



Step 2: Interchange the initial and final states of FA.



Step 3: Reverse the direction of all the transitions.



Step 4: Construct the regular grammar from this FA.

The required RLG is

$$C \rightarrow Aa$$
$$A \rightarrow Ab \mid a$$
$$S \rightarrow Ab.$$

Conversion of LLG to RLG

Left-Linear Grammar:

The left hand side of all the production in a grammar must consist of a single variable, and the right hand side consists of an optional single variable followed by one number of terminals.

All production have one of two forms

$V \rightarrow VT^*$

or

$V \rightarrow T^*$

example:

$S \rightarrow Sa \mid Ab \mid b.$

$A \rightarrow a \mid b.$

Right Linear Grammar:

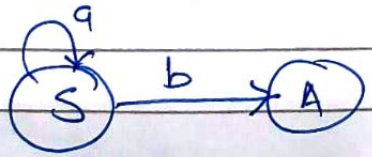
The Left hand side of all the production in a grammar should have a single variable & the right hand side consists of any number of terminals (number of T) optionally followed by a single variable.

$V \rightarrow T^*V$

or

$V \rightarrow T^*.$

ex: $S \rightarrow aS \mid bA$



Method: To convert LLG to RLG

1. Convert the given left linear grammar to FA
2. Interchange the initial and final states of FA.
3. Reverse the directions of all the transitions.
4. Construct the regular grammar from this FA.

Example 1.

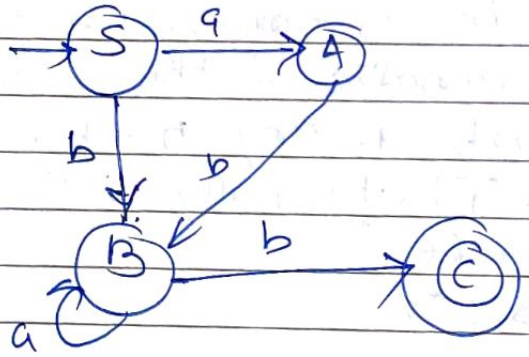
Convert the following LLG to RLG

$S \rightarrow Aa \mid Bb$

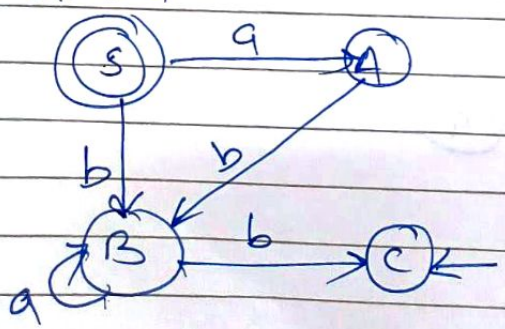
$A \rightarrow Bb$

$B \rightarrow Ba \mid b$.

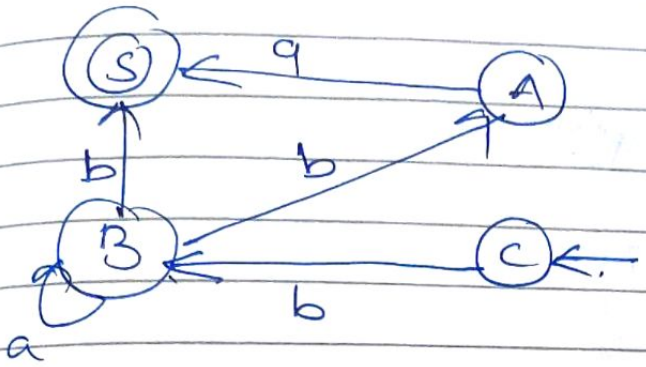
Step 1: convert the given Left linear grammar to FA.



Step 2: Interchange the initial & final states of FA.



step 3 : Reverse the direction of all the transition.



step 4 : Construct the regular grammar from this FA

The required RLG is

$C \rightarrow bB$

$B \rightarrow aB \mid bA \mid b$

$A \rightarrow a.$

Example 2

Ex. 2. Convert the following LLG to RLG.

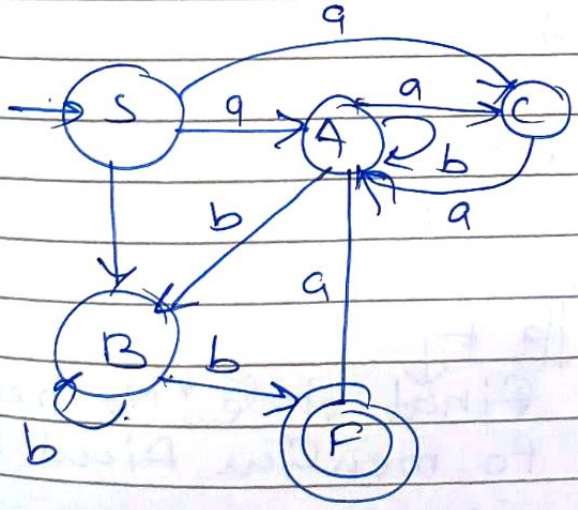
$S \rightarrow Ca \mid Aa \mid Bb$

$A \rightarrow Ab \mid Ca \mid Bb \mid a$

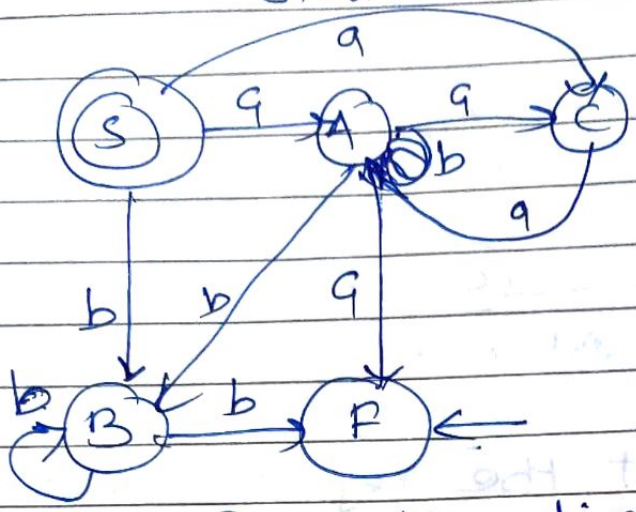
$B \rightarrow Bb \mid b$

$C \rightarrow Aa.$

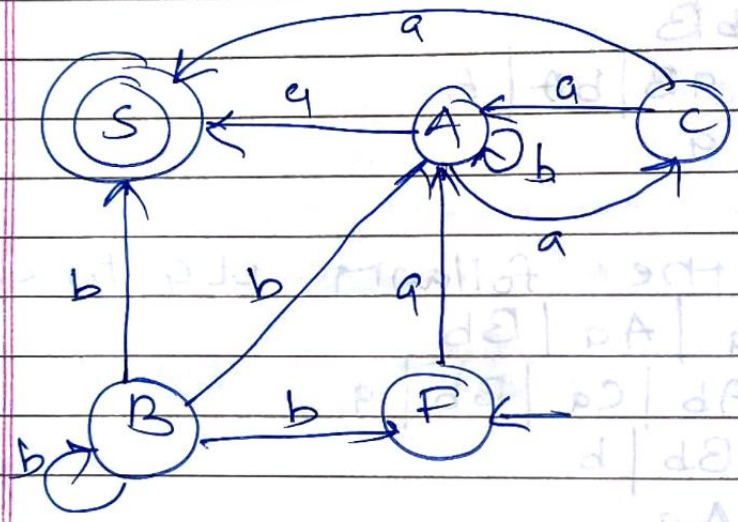
step 1. Convert LLG to FA



Step 2: Interchange the initial & Final states of FA.



step 3: Reverse the directions of all the transactions.



step 4: construct Regular grammar from this FA.

The required RLG is

$S \rightarrow aA \mid bB$

$A \rightarrow bA \mid aC \mid a$

$B \rightarrow bB \mid bA \mid b$

$C \rightarrow aA \mid a$

Final state: No need to mention final state Here.

Construction of FA from Grammar

Finite Automata:

A FA is collection of 5 tuple $(Q, \Sigma, q_0, F, \delta)$ where,

Q – Finite set of states $Q \neq \phi$
Σ – input Alphabet
q_0 – Initial state $q_0 \in Q$
F is Final state
δ is Transition function $Q \times \Sigma \rightarrow Q$.

Grammar:

A grammar G can be formally written as a 4 tuple (V, T, P, S) where,
V is a set of variables or Non terminal symbols
T or Σ is a set of Terminal symbols
P is Production rules for Terminals and Non-Terminals.
– A production rule has the form $\alpha \rightarrow \beta$, where α and β are strings on $\{V \cup \Sigma\}$ and least one symbol of α belongs to V. | union
S is a special variable called the – start symbol.

Rules:

(1) Include edge $\delta(q_i, a) = q_j$, if $A_i \to aA_j$ exists.

OR

Include an edge state $q_i \to q_j$ with an edge label 'a', if production $A_i \to aA_j$ exists in the grammar.

(2) Include edge $\delta(q_i, a) = q_f$ if $A_i \to a$ exists.

OR

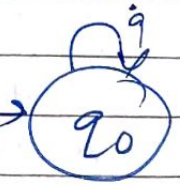
Include an edge state $q_i \to q_f$ with an edge label 'a', if production $A_i \to a$ exists in grammar.

Here 'a' can be ϵ, in that case q_f is final state without any transition.

Example: Construct a FA recognizing L(G) where G is the grammer A0 with starting symbol.

$A_0 \to aA_0 \mid bA_1 \mid b$

$A_1 \to aA_1 \mid bA_0 \mid a$



$A_0 \to aA_0 = \delta(q_0, a) = q_0$

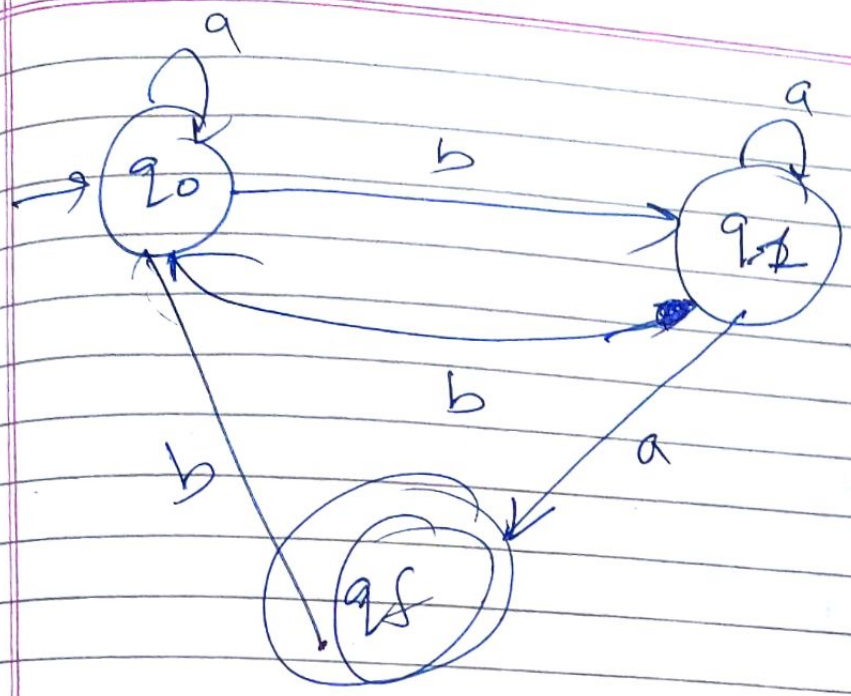
$A_0 \to bA_1 = \delta(q_0, b) = q_1$

$A_1 \to aA_1 = \delta(q_1, a) = q_1$

$A_1 \to bA_0 = \delta(q_1, b) = q_0$

$A_0 \to b = \delta(q_0, b) = q_f$

$A_1 \to a = \delta(q_1, a) = q_f$



The required Finite Automata.

$Q = \{q_0, q_1, q_f\}$

$\Sigma = \{a, b\}$

$q_0 = q_0$

$\{\delta(q_0, a) = q_0,\ \delta(q_0, b) = q_1,\ \delta(q_1, a) = q_1,$
$\delta(q_1, b) = q_0,\ \delta(q_0, b) = q_f,\ \delta(q_1, a) = q_f\}$

Grammar (V, T, P, S)

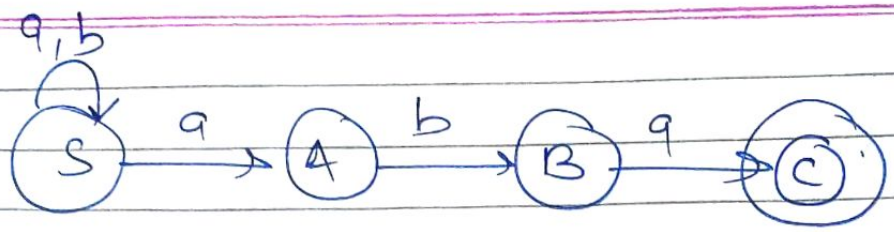
FA $(Q, \Sigma, q_0, f, \delta)$

$S = q_0 \quad V = Q \quad T = \Sigma \quad P = \delta$

---

ex:2 Construct a FA recognizing $L(G)$ where G is the grammar,

$S \to aS \mid bS \mid aA$	$S \to aS = \delta(S, a) = S$
$A \to bB$	$S \to bS = \delta(S, b) = S$
$B \to aC$	$S \to aA = \delta(S, a) = A$
$C \to \epsilon$	$A \to bB = \delta(A, b) = B$
	$B \to aC = \delta(B, a) = C$
	$C \to \epsilon$ = C is final state.



$M = (\{S, A, B, C\}, \{a, b\}, \delta, \{C\})$

How to convert Regular Grammar to Finite Automata ?

Ex°1

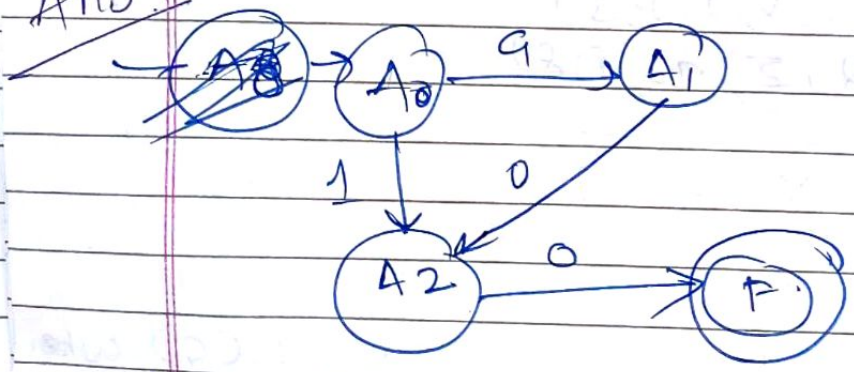
RG to FA.

Given, $V = \{A0, A1, A2\}$

$T = \{0, 1\}$

Production Rule
- $A0 \rightarrow 0A1$
- $A0 \rightarrow 1A2$
- $A1 \rightarrow 0A2$
- $A2 \rightarrow 0$ - means after giving zero we reach to final state, Not mandatory to mention.

And

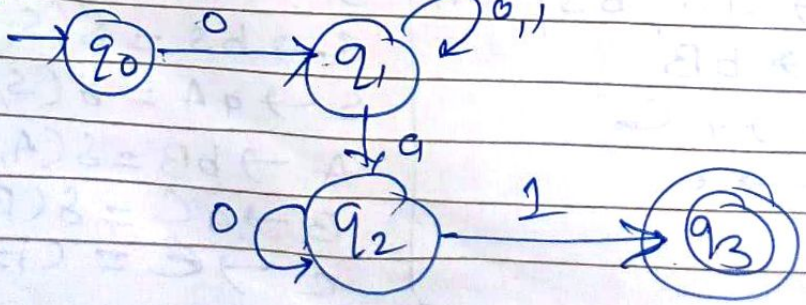


If you want we can mention. It is to be assum

Ex°1

Convert FA to RG

Given FA:



Sol?

$V = \{q_0, q_1, q_2, q_3\}$

$T = \{0, 1\}$

Production Rule:

$q_0 \rightarrow 0q_1$

$q_1 \rightarrow 0q_1$

$q_1 \rightarrow 1q_1$

$q_2 \rightarrow 0q_2$

$q_2 \rightarrow 0q_2$

$q_2 \rightarrow 1q_3$

$q_2 \rightarrow 1$

RG = $q_1 \rightarrow 0q_1 \mid 1q_1 \mid 0q_2$

~~Conversion of RE to RG~~

Constructing an Equivalent Regular Grammar from a Regular Expression.

conversion of RE & RG

Def'd Regular Expression & the Theorems.

- For any Regular Expression 'e', there exists a regular Grammar G recognising exactly the language described by 'e'.

The basic Idea is.

1. If the regular Expression is simply 0, we can show that G, with no production rules, is an equivalent regular grammar.

RE – Regular Expression
RG – Regular Grammar

(2) If the RE is simply ∈, we can show that G, with one production rule S → ∈ (where S is start symbol) is an equivalent regular Grammar.

(3) If RE is simply 'a' ('a' being any terminal), we can show that G, with one product-Rule S → a is an equivalent RG. (where S is start symbol)

(4) If RE is of form e+f (where e & f both are the regular Expressions, Use the
→ Use the Algorithm to construct two regular grammars G1 & G2 equivalent to e and f respectively
⇒ Now use regular Grammar Union on the two grammars to construct a regular grammar G which is equivalent to e+f.

(5) If the RE is of form ef (e & f both REs)
⇒ construct two RGs G1 & G2 equivalent to e & f respectively.
⇒ Use Regular Grammar concatenation on G1 & G2 which is equivalent to ef.

(6) If the RE is in the form of e^*, (where e is Regular Expression)
⇒ use to find equivalent RG.
⇒ use RG Kleen start closure to construct RG G^1 which is equivalent to e^*

(7) If the RE is in the form of e^+ to find equivalent RG,
⇒ use RG Kleen plus closure to construct a RG G^1 which is equivalent e^+.

Examples :-

- We describe Regular Grammar by just stating their production rules.
- Non terminal symbol appearing on left-hand side of the first production is assumed to be the start symbol.

eg:1

RE = $(a+b)^* a$.

First: $a+b$ we create two RGs.

$S1 \rightarrow a$

$S1 \rightarrow b$

where S1 & S2 are start symbol & these grammar recognise the regular expressions a & b respectively.

Apply Union transformation for regular grammar to get:

$S3 \rightarrow a \mid b$ — RG $\Rightarrow S3 \rightarrow a+b$

$S1 \rightarrow a$.

$S2 \rightarrow b$.

where S3 is start symbol.

Grammar recognises $a+b$.

Second step: Now we consider $(a+b)^*$.

we ~~have~~ already have a regular grammar for $(a+b)$.

- Now we apply the kleen star transformation on the regular Grammar

$S4 \rightarrow a \mid b \mid \epsilon$

$S3 \rightarrow a \mid b$

$S1 \rightarrow aS3$

$S2 \rightarrow bS3$

where S4 is start symbol.

Third Recall that we need regular Grammar that recognises $(a+b)^*a$,
- We thus consider again the regular Expression $a.A$
- Again we create a regular grammar that describes the language:

$$S5 \rightarrow a$$

where S5 is the start symbol,

Fourth
- Now, we construct the concatenation of the regular grammar describing $(a+b)^*$ together with this one.

→ we simply apply the transformation that concatenates two regular grammars to:

$S4 \rightarrow a \mid b \mid \epsilon$
$S3 \rightarrow aS5 \mid bS5$
$S1 \rightarrow aS3$
$S2 \rightarrow bS3$
$S5 \rightarrow a.$

equivalent to $(a+b)^*a$.

where S4 is the start symbol,

Example 2
RE2 $= (a+1)^*$. Construct Regular grammar for the RE,

$S \rightarrow G$
$S1 \rightarrow a$
$S2 \rightarrow \epsilon$
S1 & S2 Non-terminals,

Apply Union for RG:

$S3 \rightarrow a \mid \epsilon$
$S1 \rightarrow a$
$S2 \rightarrow \epsilon$

where S3 start-symbol.

- Now consider (a+b)*
- we already have a RG for (a+b)
- So we can apply the kleen star information on the regular grammer. Since the grammer is not ε-free,

we first have to make the grammer ε free.

$S4 \rightarrow 9\varepsilon$

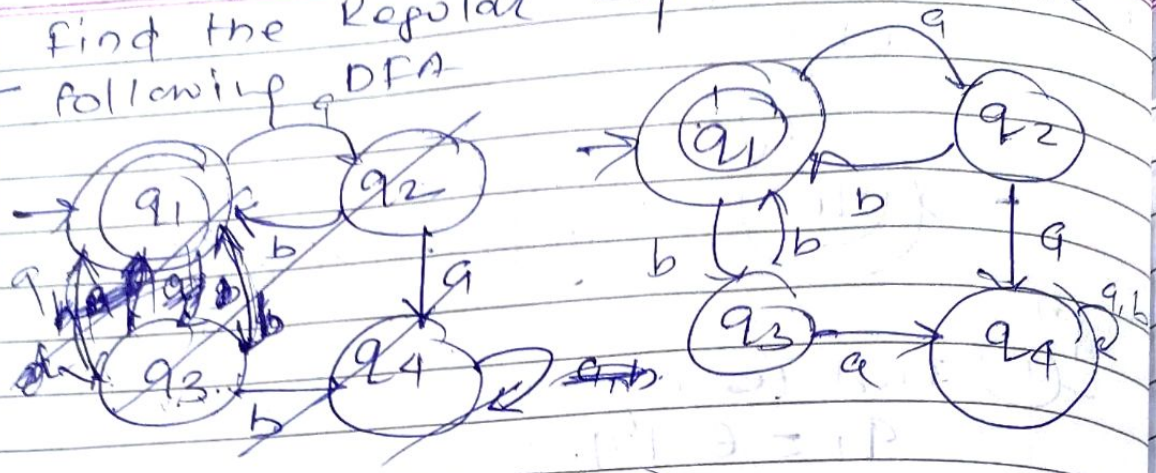
$S3 \rightarrow 9$

$S1 \rightarrow 9.$

where S4 start symbol.

END OF UNIT 2.

ex:4 Find the Regular Expression for the following DFA



Sol?

$q_1 = \epsilon + q_2 b + q_3 b$ — (1)

$q_2 = q_1 a$ — (2)

$q_3 = q_1 b$ — (3)

$q_4 = q_2 a + q_3 b + q_4 a + q_4 b$ — (4)

$\Rightarrow q_1 = \epsilon + q_2 b + q_3 a$

putting the values of q_2 and q_3 from (2) & (3)

$q_1 = \epsilon + q_1 ab + q_1 ba$

$q_1 = \epsilon + q_1 (ab + ba)$

$R_1 \quad Q \quad R \quad P$

$R = Q + RP$

$R = QP^*$

By Arden's Theorem.

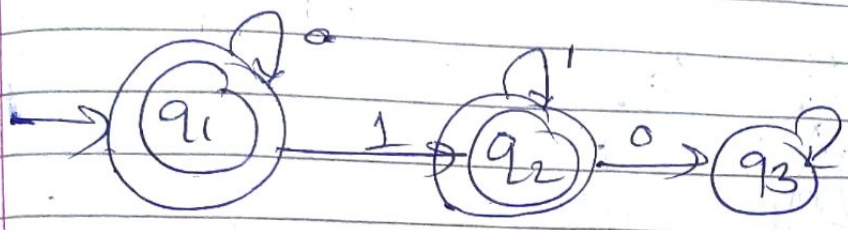
$q_1 = \epsilon (ab + ba)^*$

$\epsilon R = R$

$q_1 = (ab + ba)^*$

q_1 is final state, Hence we get the expression for final state so this $*$ is the regular Expression we got.

ex:5 Find Regular Expression for the following DFA.



Soln

(1) Find the equations

$q_1 = \epsilon + q_1 0 \longrightarrow (1)$

$q_2 = q_1 1 + q_2 1 \longrightarrow (2)$

$q_3 = q_2 0 + q_3 0 + q_3 1 \longrightarrow (3)$

Find state (q_1)

↳ $q_1 = \epsilon + q_1 0$

$R = Q + RP$

$R = Qp^*$ Aden's theorem

By Identities.

$\epsilon R = R$

$q_1 = \epsilon \cdot 0^*$

$q_1 = 0^* \longrightarrow (4)$

One more find state. (q_2)

↳ $q_2 = q_1 1 + q_2 1$

$= 0^* 1 + q_2 1$ put the value of q_1 from (4).

$R = Q + RP$

$R = Qp^*$ Aden's theorem

$q_2 = 0^* 1 (1)^*$

R = union of both the final states.

$= 0^* + 0^* 1 1^*$ $\quad \epsilon + RR^* = R^*$

$= 0^* (\epsilon + 11^*)^*$

$= 0^* 1^* \rightarrow$ Regular Expression for DFA.